

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 7 Laboratory Sheet

Modeling Languages II: The Mathematical Programming View

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours
Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1, a text editor, and network access for a one-time package install

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction — Why It Matters

Every model you have built so far this course answers "is there a solution, or here are all of them." This lab hands you the toolkit that instead asks "which solution is provably the *best*, and prove no better one exists" — the same question a factory manager asks about a production plan. In the next two hours you write your first `gurobipy` models, hand-trace the exact search a MIP solver runs internally before you ever call it, catch a genuinely broken assignment model by running it yourself, and close by building one such model completely from scratch.

1 • Pre-instruction — Prior Knowledge

This sheet exercises all four units of this week's chapter: linear and integer programs and LP relaxation (Unit 1), branch-and-bound (Unit 2), first `gurobipy` models for knapsack, assignment and production planning (Unit 3), and the CP-SAT vs. `gurobipy` comparison (Unit 4). Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Adding gurobipy to the Course's Shared Environment

Step 1: Install gurobipy into the shared virtual environment

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` (created in Week 1, already carrying `ortools`, `minizinc` and `pytest` from earlier weeks) active, you will install this week's one new package, `gurobipy`, at the exact pinned version the whole course uses.

Why? Every `gurobipy` model this week is checked against a real, captured solver run; pinning `gurobipy` to the exact version this course uses (Gurobi 12.0.3) is what makes those captured outputs reproducible on your own machine.

Step 1 (continued)

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ pip install gurobipy==12.0.3
```

Expected result: `pip` downloads and installs `gurobipy` and finishes with a line confirming it installed.

Expected Output

```
Collecting gurobipy==12.0.3
  Using cached gurobipy-12.0.3-cp312-cp312-macosx_10_9_universal2.whl.metadata
  ↪  (16 kB)
Using cached gurobipy-12.0.3-cp312-cp312-macosx_10_9_universal2.whl (12.2 MB)
Installing collected packages: gurobipy
Successfully installed gurobipy-12.0.3
```

Tip

No `gurobi.lic` licence file is needed for this course: installing the `gurobipy` package alone is enough. With no licence file present, `gurobipy` falls back to a built-in *restricted* licence (solves models up to 2000 variables and 2000 constraints) — exactly what Unit 3 shows you happening on a real run.

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead.

Step 2: Verify the installed version

What will you do? With the environment still active, you will print the installed `gurobipy` version to confirm the pinned install succeeded.

Why? A silent, successful `pip install` does not by itself prove the right version landed on your `PATH`; this check catches a stale or wrong installation before it costs you time later in the sheet.

Terminal

```
$ python -c "import gurobipy; print(gurobipy.__version__)"
```

Expected result: The command prints `12.0.3`.

Expected Output

```
12.0.3
```

Step 2 (continued)

Common Pitfall

Setting the environment variable `GRB_LICENSE_FILE` to a non-existent path, hoping to "point at nothing" and force the restricted licence, does *not* work — `gurobipy` raises an error trying to read that path instead of falling back gracefully. Leave `GRB_LICENSE_FILE` unset entirely; `gurobipy` only falls back to the restricted licence when the variable is *absent*, not when it points at a missing file.

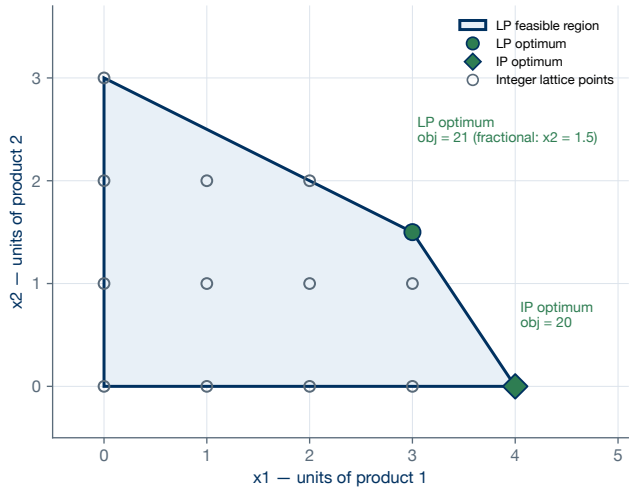
2 Unit 1 — Linear and Integer Programs, and LP Relaxation

(*See Course Notes Week 7, Unit 1 — Linear and Integer Programs, and LP Relaxation.*)

Step 3: Formulate a new knapsack instance as an IP

What will you do? A 0/1 knapsack has capacity 8 and three items: X (weight 3, value 5), Y (weight 4, value 6), Z (weight 5, value 7) — a fresh instance, not the A/B/C instance the chapter's Unit 1 worked by hand. On paper, write its IP formulation (objective and constraint, exactly as Unit 1's worked example does for A/B/C), then enumerate the 8 subsets by hand and state the true integer optimum.

Why? Writing out a second knapsack IP from a plain description, without a paper model handed to you first, is what turns following Unit 1's A/B/C derivation into formulating one yourself — exactly the practice the chapter's own Unit 1 task rehearses on this same A/B/C instance, applied here to numbers you have not seen worked out yet.



Expected result: IP: maximize $5x_X + 6x_Y + 7x_Z$ subject to $3x_X + 4x_Y + 5x_Z \leq 8$, $x_X, x_Y, x_Z \in \{0, 1\}$. Checking all 8 subsets against the weight-8 capacity, the two feasible candidates worth comparing are $\{X, Y\}$ (weight 7, value 11) and $\{X, Z\}$ (weight 8, value **12**); every other feasible subset scores lower, and $\{Y, Z\}$ and $\{X, Y, Z\}$ both exceed the capacity. The true integer optimum is $\{X, Z\}$, value 12, using the full capacity of 8.

Step 3 (continued)

Tip

The figure above is Unit 1's two-variable feasible-region illustration (a separate, deliberately simple production-style toy, not this knapsack instance) — use it only to recall what "feasible region" and "LP optimum at a vertex" look like geometrically before you work this three-item instance by hand.

Step 4: Compute the LP relaxation of the X/Y/Z instance by hand

What will you do? Drop the integrality restriction on the same X/Y/Z instance — replace $x_X, x_Y, x_Z \in \{0, 1\}$ with $0 \le x_X, x_Y, x_Z \le 1$, keeping the objective and constraint identical — and compute the relaxation's optimum by hand with the greedy value-per-weight rule, stating which variable ends up fractional.

Why? This is the exact hand-computation Unit 1's own worked example and its practice task both walk through on A/B/C; repeating it on a fresh instance is what confirms you can apply the greedy rule yourself, not just follow along with someone else's arithmetic — and it sets up the branch-and-bound trace the next section asks you to run on these same numbers.

Expected result: Ratios: X $= 5/3 \approx 1.667$, Y $= 6/4 = 1.5$, Z $= 7/5 = 1.4$ — so fill in the order X, Y, Z. Take X fully (weight 3, value 5, capacity 5 left); take Y fully (weight 4, value 6, capacity 1 left); Z's remaining weight of 5 no longer fits whole, so take the fraction that exactly uses the last unit of capacity, $x_Z = 1/5 = 0.2$ (weight 1, value 1.4). The relaxed optimum is $5 + 6 + 1.4 =$ **12.4** — strictly above the true integer optimum of 12 (previous step), and the fractional variable is x_Z.

Checkpoint — verify before moving on

You have formulated a second knapsack IP entirely from a plain description, enumerated its 8 subsets by hand to find the true optimum (12), and computed its LP relaxation by hand (12.4, fractional on x_Z). If both match the `Expected result` boxes above, you are ready to see exactly how a solver closes that 12.4-to-12 gap.

3 Unit 2 — Branch-and-Bound and Why MIP Works

(*See Course Notes Week 7, Unit 2 — Branch-and-Bound and Why MIP Works.*)

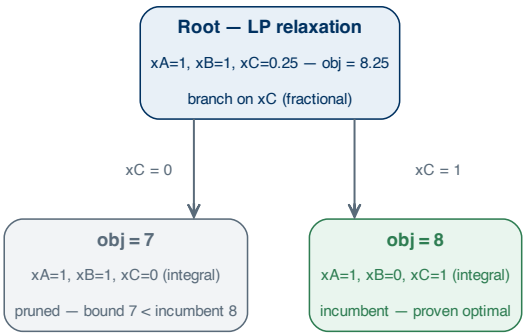
Step 5: Hand-trace branch-and-bound on the X/Y/Z instance

What will you do? Continuing the previous section's X/Y/Z knapsack instance (capacity 8; root relaxation 12.4, fractional on x_Z), branch on x_Z exactly the way Unit 2's own A/B/C trace branches on x_C: solve the LP relaxation of the $x_Z = 0$ child and of the $x_Z = 1$ child by hand, state which child's value becomes the incumbent, which child gets pruned and why, and the final proven optimum.

Why? Unit 2's own hand trace works this exact procedure on the A/B/C instance; repeating it here, on numbers you have not seen the answer to yet, is what turns following the bounding argument into running it yourself — and it is the same bounding argument, applied by hand, that lets Unit 3's `gurobipy` runs report a result as *provably* optimal rather than merely the best one tried.

Step 5 (continued)

Expected result: Child $x_Z = 0$: maximize $5x_X + 6x_Y$ subject to $3x_X + 4x_Y \leq 8$, $0 \leq x_X, x_Y \leq 1$; greedy (X's ratio 1.667 still above Y's 1.5) gives $x_X = 1, x_Y = 1$ — already integral, weight $3 + 4 = 7 \leq 8$, value $5 + 6 = \mathbf{11}$. Child $x_Z = 1$: fixing $x_Z = 1$ uses weight 5 and value 7, leaving capacity $8 - 5 = 3$; the relaxation maximizes $5x_X + 6x_Y$ subject to $3x_X + 4x_Y \leq 3$, $0 \leq x_X, x_Y \leq 1$; greedy (X's weight of 3 exactly fits the remaining capacity, Y no longer fits at all) gives $x_X = 1, x_Y = 0$ — already integral, weight $3 + 5 = 8$, value $5 + 7 = \mathbf{12}$. The second child's integral value, 12, becomes the incumbent; the first child's value, 11, is strictly below that incumbent, so it is pruned — no further branching is needed anywhere. The proven optimum is **12**, matching the previous section's by-hand enumeration of $\{X, Z\}$ exactly.



Tip

The figure above draws Unit 2's own A/B/C trace, not this X/Y/Z one — use it as a template for the three-node shape (root, one pruned child, one incumbent child) your own X/Y/Z trace should have, then sketch your own three-node tree with the 12.4/11/12 values on paper.

4 Unit 3 — First Models in gurobipy

(See Course Notes Week 7, Unit 3 — First Models in gurobipy: Assignment, Knapsack, and Production Planning.)

Step 6: Run knapsack.py: the IP and its LP relaxation, for real

What will you do? You will run `knapsack.py`, which solves Unit 1's original A/B/C knapsack instance (capacity 6) twice — once as a binary IP, once with the same three variables relaxed to `GRB.CONTINUOUS` — and check both printed results against the values you already know from Unit 1's own by-hand work.

Why? This closes the loop Unit 1 opened on paper: the IP's optimal value (8) and the relaxation's optimal value (8.25, fractional on C) are not new numbers here — they are a real solver confirming a calculation the chapter's Unit 1 already walked you through by hand, the same way you just confirmed your own X/Y/Z numbers on paper before trusting them.

The `GRB.BINARY` variables and the shared capacity constraint:

```
model = gp.Model("knapsack_ip")
x = {item: model.addVar(vtype=GRB.BINARY, name=item) for item in ITEMS}
```

Step 6 (continued)

```
model.addConstr(
    gp.quicksum(WEIGHT[item] * x[item] for item in ITEMS) <= CAPACITY,
    name="capacity",
)
```

Terminal

```
$ cd hafta07
$ python modeller/knapsack.py
```

Expected result: A status of `OPTIMAL` with optimal value 8.0 (items A and C packed, B left out), followed by the relaxation's value 8.25 with C fractional at 0.25 — exactly Unit 1's hand-computed A/B/C numbers.

Expected Output

```
Status: OPTIMAL
Optimal value: 8.0
  A: 1.0
  B: 0.0
  C: 1.0
```

Expected Output

```
Relaxed optimal value: 8.25
  A: 1.0
  B: 1.0
  C: 0.25
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta07` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta07`.

Step 7: Run assignment.py: the worked four-worker/four-task problem

What will you do? You will run `assignment.py`, Unit 3's complete four-worker (W1–W4) / four-task (Ta–Td) model, and check its printed pairing and total cost against the worked cost matrix's cheap diagonal (4, 3, 2, 5) and uniform off-diagonal cost of 9.

Why? This is Unit 3's complete, working reference: both constraint families (every worker does exactly one task, every task gets exactly one worker) are already present here, so running it first gives you a correct model's output to compare the next step's deliberately incomplete model against.

Two constraint families, both present:

```
for worker in WORKERS:
    model.addConstr(gp.quicksum(x[worker, task] for task in TASKS) == 1)
for task in TASKS:
    model.addConstr(gp.quicksum(x[worker, task] for worker in WORKERS) == 1)
```

Step 7 (continued)

Terminal

```
$ python modeller/assignment.py
```

Expected result: The unique optimal assignment is the cost matrix's diagonal, total cost 14.

Expected Output

```
W1 -> Ta
W2 -> Tb
W3 -> Tc
W4 -> Td
Total cost: 14.0
```

Step 8: Run the incomplete assignment_lab.py and find the bug

What will you do? A second, smaller assignment problem — three workers (S1–S3), three tasks (J1–J3) — lives in `kod_ornekleri/hafta07/lab/assignment_lab.py`. Its `build_model` function declares the binary variables and only the "every worker does exactly one task" constraint family; the "every task gets exactly one worker" family is missing, marked with a `TODO` comment. Run the file exactly as given and inspect the printed pairing carefully.

Why? Running the broken model before fixing it, on a cost matrix chosen so the bug actually changes the answer, is what makes "a missing constraint family" concrete rather than abstract — unlike Unit 3's own worked assignment example (where, by coincidence, every worker's individually cheapest task already happens to be distinct), this instance's cheapest tasks collide, so the missing constraint produces a visibly invalid result.

The variables and the incomplete constraint block:

```
model = gp.Model("assignment_lab")
x = model.addVars(WORKERS, TASKS, vtype=GRB.BINARY, name="x")
```

```
for worker in WORKERS:
    model.addConstr(gp.quicksum(x[worker, task] for task in TASKS) == 1)
# TODO: add the missing "every task gets exactly one worker" family
# here -- one constraint per task in TASKS.
```

Terminal

```
$ python lab/assignment_lab.py
```

Expected result: A status of `OPTIMAL` and a total cost of 6.0 — but look closely at the pairing: task J3 is used *twice* (by S2 *and* S3), while task J1 is not used at all. This is not a valid assignment; it is only "optimal" because the model never told the solver that each task may be used once.

Expected Output

```
S1 -> J2
S2 -> J3
S3 -> J3
Total cost: 6.0
```

Step 8 (continued)

Common Pitfall

A total cost of 6.0 looks *better* than the 9.0 you will find once the model is fixed in the next step — it is tempting to assume a lower number must be the better answer. It is not: 6.0 is the cost of an assignment that double-books J3 and leaves J1 unstaffed, which is not a valid solution to the problem at all. A missing constraint does not just risk `INFEASIBLE`; it can just as easily produce a confidently printed, cheaper-looking, wrong answer.

Step 9: Add the missing constraint family and verify the fix

What will you do? In `lab/assignment_lab.py`, where the `TODO` comment says to, add the missing constraint family: one constraint per task, stating that exactly one worker is assigned to it. Then re-run the script.

Why? This is Unit 3's own "Now You Try" pattern — completing a model that is missing exactly this one constraint family — but here you run your own fix against a real solver immediately, rather than only checking it against a written answer, closing the loop from describing the missing pattern to fixing a model and watching the wrong answer disappear.

Terminal

```
$ python lab/assignment_lab.py
```

Expected result: A valid, unique assignment with every task used exactly once: S1 to J2, S2 to J1, S3 to J3, total cost **9.0** — verifiable by hand, since this is the cheapest of the six possible complete pairings of three workers to three tasks.

Expected Output

```
S1 -> J2
S2 -> J1
S3 -> J3
Total cost: 9.0
```

Tip

The missing pattern mirrors `assignment.py`'s own second constraint family, just reindexed by task instead of by worker: `for task in TASKS: model.addConstr(gp.quicksum(x[worker, task] for worker in WORKERS) == 1)`.

Checkpoint — verify before moving on

`knapsack.py` confirmed Unit 1's A/B/C numbers (8 and 8.25) on a real solver, `assignment.py` reproduced the worked diagonal solution (cost 14), and your own fix to `assignment_lab.py` turned an invalid, double-booked "optimal" answer (cost 6.0) into a genuinely valid one (cost 9.0). You have now both run a correct model and repaired a broken one.

Step 10: Run production_planning.py and the week's test gate

What will you do? You will run `production_planning.py` — a brand-new problem this week, three products competing for two shared resource budgets — and then run the full `pytest` gate for this week's code, confirming every test passes.

Why? This is the one model this week whose exact optimum is not computed by hand anywhere in the chapter, so running it is your only source for these numbers; it is also the first solve this sheet shows you with the restricted-licence banner printed in full, which every `gurobipy` call you have made so far in this sheet has also triggered, just without you being asked to look at the top of the log until now.

Terminal

```
$ python modeller/production_planning.py
$ python -m pytest -q
```

Expected result: The solver log opens with the restricted-licence banner, followed by the solve; the plan and profit reported are 9 units of product P1, 0 of P2, 13 of P3, total profit 620.0. `pytest` reports every test passing.

Expected Output

```
Restricted license - for non-production use only - expires 2026-11-23
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin 25.6.0
↪  25G83)
```

Expected Output

```
P1: 9.0
P2: 0.0
P3: 13.0
Total profit: 620.0
```

Expected Output

```
....                                                                [100%]
4 passed in 0.02s
```

Caution

The restricted-licence banner is expected, unmodified output, not an error — it is exactly what `gurobipy` prints whenever no licence file grants it a full commercial licence, and it says nothing about whether your model solved correctly. It only refuses models above 2000 variables or 2000 constraints; every model in this course stays at single-digit counts, far under that limit.

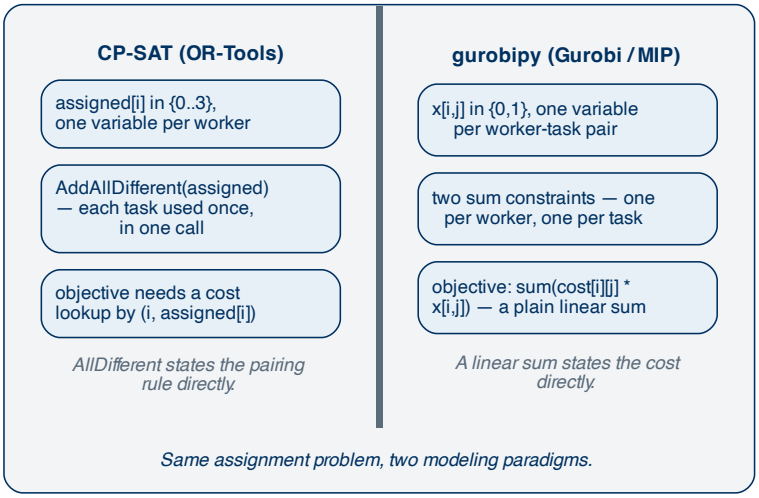
5 Unit 4 — CP vs MIP: The Same Problem, Two Ways

(See Course Notes Week 7, Unit 4 — CP vs MIP: The Same Problem, Two Ways.)

Step 11: Sketch a CP-SAT model of your own assignment instance

What will you do? Using the three-worker/three-task instance from `assignment_lab.py` (S1–S3, J1–J3, the cost matrix from the earlier steps), sketch — in words and API names only, no code — how CP-SAT would model it: which decision variables, which single global constraint states "every task is used by exactly one worker," and what would have to stand in for the cost objective. Then state which of the two models — this CP-SAT sketch or your completed `gurobipy` model — more directly expresses the pairing constraint as a single piece of the model, and which more directly expresses the cost to minimize.

Why? Unit 4 works this exact comparison on the chapter's own four-worker instance; repeating the same reasoning on your own three-worker instance, whose numbers you already hand-verified in the previous section, is what turns following Unit 4's comparison into applying the same rule of thumb to a problem of your own.



Expected result: CP-SAT sketch: one integer variable `assigned[i]` per worker in {S1, S2, S3}, domain {0, 1, 2} standing for the task index; a single `model.AddAllDifferent(assigned)` call directly captures "every task is used by exactly one worker." The cost objective would need an element- or table-lookup-style construct (or auxiliary cost variables), since the cost of worker i's choice depends on both i and the *value* `assigned[i]` takes. `AllDifferent` states the pairing idea more directly, in one call, than `gurobipy`'s two families of sum constraints; your completed `gurobipy` model's `sum(cost[worker][task] * x[worker, task] ...)` states the cost objective more directly than CP-SAT's lookup-based construction would — each paradigm wins on a different half of the same problem, exactly Unit 4's rule of thumb applied to a fresh instance.

Checkpoint — verify before moving on

You have applied Unit 4's CP-SAT/`gurobipy` comparison to a problem instance of your own, not just the chapter's worked example. Every unit's ideas — LP relaxation, branch-and-bound, running `gurobipy` code, and comparing it against CP-SAT — have now been exercised at least twice each: once on the chapter's own numbers, once on numbers this sheet gave you fresh.

6 Capstone — Build a Complete Model From Scratch

Step 12: Build a travel-budget MIP entirely from scratch

What will you do? Recall Unit 1's travel-budget analogy: a fully divisible line item (meals) alongside an all-or-nothing line item (conference registration) in one budget. Turn that analogy into a real, running `gurobipy` model, written entirely by yourself in a new file (no starter file this time): a research travel budget of 700 currency units; a binary decision `register` for conference registration, costing 400 and worth 10 units of value; a continuous decision `meal_units` for meal spending, bounded between 0 and 3 units, each unit costing 100 and worth 2 units of value. Maximize total value subject to the combined budget constraint $400 \cdot$ `register` $+ 100 \cdot$ `meal_units` ≤ 700, then verify your printed result against the value you compute by hand below *before* you run it.

Why? This is this week's integrative step: translating a paper LP/MIP formulation — the exact travel-budget analogy Unit 1 used to introduce mixing continuous and binary variables in one model — into a verified, running `gurobipy` program without a paper model or a starter file handed to you first, the same pattern Week 1's capstone used for `gurobipy`'s CP-SAT counterpart, and the only model this week that mixes `GRB.CONTINUOUS` and `GRB.BINARY` in a single model.

Expected result: By hand: if `register` $= 1$ (cost 400, value 10), the remaining budget is 300, allowing `meal_units` $= \min(3, 300/100) = 3$ (cost 300, value 6) — total value $10 + 6 =$ **16**, using the full budget of 700. If `register` $= 0$, only `meal_units` $= 3$ is reachable (the upper bound, well within budget), value 6 — strictly worse. So the optimum is `register` $= 1$, `meal_units` $= 3.0$, total value **16.0**. Your own script's variable names and print statements will differ from the reference run below, but `model.ObjVal` must read 16.0 and the two decision variables' `.X` values must match 1.0 and 3.0.

Expected Output

```
Status: OPTIMAL
register: 1.0
meal_units: 3.0
Total value: 16.0
```

Tip

Build the model with the same four-piece recipe every script this week has used: `gp.Model()`, then `addVar` once with `vtype=GRB.BINARY` and once with `vtype=GRB.CONTINUOUS` (remember `lb` and `ub` for the bounded continuous variable), one `addConstr` for the budget, then `setObjective(..., GRB.MAXIMIZE)` and `optimize()`.

Common Pitfall

Forgetting the upper bound `ub=3` on `meal_units` lets the solver spend the entire 700-unit budget on meals alone (`meal_units`$= 7$, value 14) whenever `register`$= 0$ is cheaper to skip — a model that runs without error and returns a plausible-looking number is not the same as a model that matches the formulation you were asked to build; always re-check every bound you meant to state, not only every constraint.

Checkpoint — verify before moving on

Your from-scratch travel-budget model reports `register`= 1.0, `meal_units`= 3.0, total value 16.0 — matching the number you derived on paper before running it. This is the same variables-objective-constraints-solve pattern you have now built or repaired five times today, the last time entirely from a written description rather than a paper model or a starter file someone else prepared for you.

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'gurobipy'`	The virtual environment is not active in this terminal, or `gurobipy` was never installed into it	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`, then re-run `pip install gurobipy==12.0.3`
`python -c "import gurobipy"` raises an error mentioning `GRB_LICENSE_FILE`	The environment variable `GRB_LICENSE_FILE` is set and points at a file that does not exist	Run `unset GRB_LICENSE_FILE` (macOS/Linux) or remove the variable from your Windows environment settings, then try again
Your model reports `Model too large for size-limited licence`	More than 2000 variables or 2000 constraints were declared, almost always from an unintentionally large loop range	Print `model.NumVars` and `model.NumConstrs` right after building the model and compare against the small counts this week's problems expect
`python modeller/knapsack.py` prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta07` before running any script
Your fixed `assignment_lab.py` still prints a task used twice	The new `addConstr` loop was written over `WORKERS` instead of `TASKS`, so it duplicates the existing constraint family instead of adding the missing one	Re-read the missing pattern in the `ipucu` box: the loop variable must be `task in TASKS`, summing over `worker in WORKERS` inside it
`pytest -q` reports failures unrelated to your own edits	`modeller/` files were edited directly instead of the `lab/` starter file	Restore `modeller/*.py` from the course repository; only `lab/assignment_lab.py` is meant to be edited this week

5 • Follow-through **Transfer — Where Else?**

The three problems you ran today — knapsack, assignment, and production planning — are not disposable lab work: Week 10 attaches a full sensitivity report to this exact code, reusing the same `gurobipy` models unchanged, and Week 12 reuses assignment's two-constraint-family pattern for rostering. More generally, the habit you practiced five times today — state decision variables, a linear objective and linear constraints on paper first, compute or bound the answer by hand, *then* hand the whole thing to a solver — is this week's version of the single habit this entire course exists to build, now extended from "is there a solution" to "which solution is provably the best, and why can nothing beat it." Whatever LP or MIP you meet next, in a paper, in your thesis, or in a future job, the questions to ask first are the same ones this sheet

just asked: what are the decision variables, is the objective and are the constraints linear, and which variables truly need to be integer or binary rather than continuous.